

Ejercicio Complementaria - Diseño de un Volante de Inercia

```
In [49]: import numpy as np
from numpy import deg2rad, rad2deg
from sympy import symbols, pi, sin, cos, lambdify, solve, linear_eq_to_matrix, simplify
import sympy as sp
from sympy.physics.mechanics import ReferenceFrame, dynamicsymbols
from scipy.optimize import fsolve
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.animation as animation
from IPython.display import HTML
```

Paso -1 - Contexto del problema

Tenemos un mecanismo *crank-rocker* de 4 barras donde la manivela **OA** gira a velocidad constante $\omega_1 = 30 \text{ rad/s}$ (sentido antihorario), haciendo que el balancín **BC** oscile como péndulo invertido.

El problema con eso es que, como la geometría del mecanismo cambia constantemente, el torque que el motor debe aportar en cada instante varía bastante (puede ser positivo o negativo), lo que en la práctica hace que el motor trabaje mal y se pueda dañar.

La solución clásica es acoplar un **volante de inercia** al eje motor: actúa como "banco de energía" que absorbe cuando sobra torque y entrega cuando falta, aplanando la curva. El motor entonces puede operar a un torque prácticamente constante.

El plan de ataque es:

1. Cinemática (FK, velocidades, aceleraciones)
2. Cinética inversa, osea curva de torques $M_{in}(\theta_1)$
3. Diseño del volante de inercia
4. Verificación de efectividad
5. Selección del motor

Paso 0 - Establecer variables y parámetros

Primero definimos todo lo simbólico y los parámetros numéricos del mecanismo según la figura del enunciado. Las barras se asumen de **aluminio 6061** con sección transversal cuadrada de $b = 10 \text{ mm}$. La nomenclatura es:

Símbolo	Valor	Descripción
L_1	80 mm	Manivela OA

Símbolo	Valor	Descripción
L_2	240 mm	Acoplador AB
L_3	200 mm	Balancín BC
L_{4x}, L_{4y}	190, 70 mm	Tierra OC (vector)
ρ	2 700 kg/m ³	Densidad aluminio 6061
b	10 mm	Lado de la sección cuadrada
m_1	21.60 g	Masa manivela ($\rho b^2 L_1$)
m_2	64.80 g	Masa acoplador ($\rho b^2 L_2$)
m_3	54.00 g	Masa balancín ($\rho b^2 L_3$)
I_1	1.152 $\times 10^{-5}$ kg·m ²	Inercia manivela (barra delgada)
I_2	3.110 $\times 10^{-4}$ kg·m ²	Inercia acoplador (barra delgada)
I_3	1.800 $\times 10^{-4}$ kg·m ²	Inercia balancín (barra delgada)

Las masas e inercias se calculan con:

$$m_i = \rho b^2 L_i, \quad I_i = \frac{1}{12} m_i L_i^2$$

```
In [50]: # Símbolos
t, g = symbols('t g')
L1, L2, L3, L4x, L4y, m1, m2, m3, I1, I2, I3 = symbols(
    'L1 L2 L3 L4x L4y m1 m2 m3 I1 I2 I3'
)
theta1, theta2, theta3 = dynamicsymbols('theta1 theta2 theta3')

# Material: aluminio 6061
rho_al = 2700.0          # kg/m3  densidad aluminio 6061
b_sec = 0.010            # m      Lado de la seccion cuadrada (10 mm)
A_sec = b_sec**2         # m2     area de la seccion transversal

# Longitudes de cada eslabon (m)
L1_val, L2_val, L3_val = 0.08, 0.24, 0.20

# Masas: m = rho * A * L
```

```

m1_val = rho_al * A_sec * L1_val # 0.02160 kg
m2_val = rho_al * A_sec * L2_val # 0.06480 kg
m3_val = rho_al * A_sec * L3_val # 0.05400 kg

# Momentos de inercia (barra delgada respecto al CM):  $I = (1/12) m L^2$ 
I1_val = (1/12) * m1_val * L1_val**2 # 1.152e-05 kg*m^2
I2_val = (1/12) * m2_val * L2_val**2 # 3.110e-04 kg*m^2
I3_val = (1/12) * m3_val * L3_val**2 # 1.800e-04 kg*m^2

# Parámetros numéricos
params = {
    L1: L1_val, L2: L2_val, L3: L3_val,
    L4x: 0.19, L4y: 0.07,
    g: 9.81,
    m1: m1_val, m2: m2_val, m3: m3_val,
    I1: I1_val, I2: I2_val, I3: I3_val
}

# Marcos de referencia
# N = marco inercial
# O = marco de la manivela (rota theta1 respecto a N)
# A = marco del acoplador (rota theta2 respecto a N)
# B = marco del balancin (rota theta3 respecto a N)
N = ReferenceFrame('N')
O = N.orientnew('O', 'Axis', [theta1, N.z])
A = N.orientnew('A', 'Axis', [theta2, N.z])
B = N.orientnew('B', 'Axis', [theta3, N.z])

# Vectores de posición de cada eslabón
rOA = L1 * O.x # manivela: O -> A
rAB = L2 * A.x # acoplador: A -> B
rBC = L3 * B.x # balancin: B -> C
rCO = L4x * N.x + L4y * N.y # tierra: C -> O (fijo en N)

# Ecuación de lazo:  $rOA + rAB + rBC + rCO = 0$ 
eqLoop = (rOA + rAB + rBC + rCO).subs(params)

omega1_val = 30.0 # rad/s velocidad angular impuesta
alpha1_val = 0.0 # rad/s^2 velocidad constante -> aceleracion cero

print(f"Material: aluminio 6061, rho = {rho_al} kg/m3, seccion {b_sec*1000:.0f}x{b_sec*1000:.0f} mm")
print(f"m1 = {m1_val:.5f} kg m2 = {m2_val:.5f} kg m3 = {m3_val:.5f} kg")
print(f"I1 = {I1_val:.5e} kg*m2 I2 = {I2_val:.5e} kg*m2 I3 = {I3_val:.5e} kg*m2")
print("Variables definidas correctamente.")
print(f"Lazo vectorial: {eqLoop}")

```

```

Material: aluminio 6061, rho = 2700.0 kg/m3, seccion 10x10 mm
m1 = 0.02160 kg m2 = 0.06480 kg m3 = 0.05400 kg
I1 = 1.15200e-05 kg*m2 I2 = 3.11040e-04 kg*m2 I3 = 1.80000e-04 kg*m2
Variables definidas correctamente.
Lazo vectorial: 0.19*N.x + 0.07*N.y + 0.08*O.x + 0.24*A.x + 0.2*B.x

```

Paso 1 - Cinemática directa (FK)

La ecuación de lazo $rOA + rAB + rBC + rCO = 0$ nos da dos ecuaciones escalares (proyección en N.x y N.y). Dado θ_1 , resolvemos numéricamente para θ_2 y θ_3 con `fsolve` - eso es el FK.

```

In [51]: # Ecuaciones escalares del lazo
eqKin = [eqLoop.dot(N.x), eqLoop.dot(N.y)]
eqKin_fun = lambdify([theta1, theta2, theta3], eqKin)

# Puntos de interés del mecanismo
points = {

```

```

'O': 0 * N.x,
'A': rOA,
'B': rOA + rAB,
'C': -rCO          # C = -rCO pues rCO va de C hacia O
}

points_fun = {
    k: lambdaify(
        [theta1, theta2, theta3],
        [v.dot(N.x).subs(params), v.dot(N.y).subs(params)]
    )
    for k, v in points.items()
}

```

```

In [52]: def plotMechanism(joint_values, ax=None):
    """Dibuja el mecanismo de 4 barras dados [θ1, θ2, θ3]."""
    Po = points_fun['O'](*joint_values)
    Pa = points_fun['A'](*joint_values)
    Pb = points_fun['B'](*joint_values)
    Pc = points_fun['C'](*joint_values)

    if ax is None:
        ax = plt.gca()

    # Eslabones
    ax.plot([Po[0], Pa[0]], [Po[1], Pa[1]], 'r-', lw=2.5, label='OA (manivela)')
    ax.plot([Pa[0], Pb[0]], [Pa[1], Pb[1]], 'b-', lw=2.5, label='AB (acoplador)')
    ax.plot([Pb[0], Pc[0]], [Pb[1], Pc[1]], 'g-', lw=2.5, label='BC (balancín)')
    ax.plot([Pc[0], Po[0]], [Pc[1], Po[1]], 'k--', lw=1.5, label='CO (tierra)')

    # Juntas
    ax.plot([Po[0], Pa[0], Pb[0], Pc[0]],
            [Po[1], Pa[1], Pb[1], Pc[1]], 'ko', ms=6, zorder=5)

    ax.set_xlim(-0.35, 0.35)
    ax.set_ylim(-0.35, 0.35)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.4)

```

Pequeña prueba visual antes de animarlo

Le paso un ángulo de prueba para verificar que el mecanismo cierra bien. Si los eslabones quedan desconectados algo está mal en las ecuaciones, entonces sirve como sanity check.

```

In [53]: # FK: resolver theta2, theta3 dado theta1
z0 = [deg2rad(0), deg2rad(0)] # initial guess - ojo: cambia si el mecanismo no converge

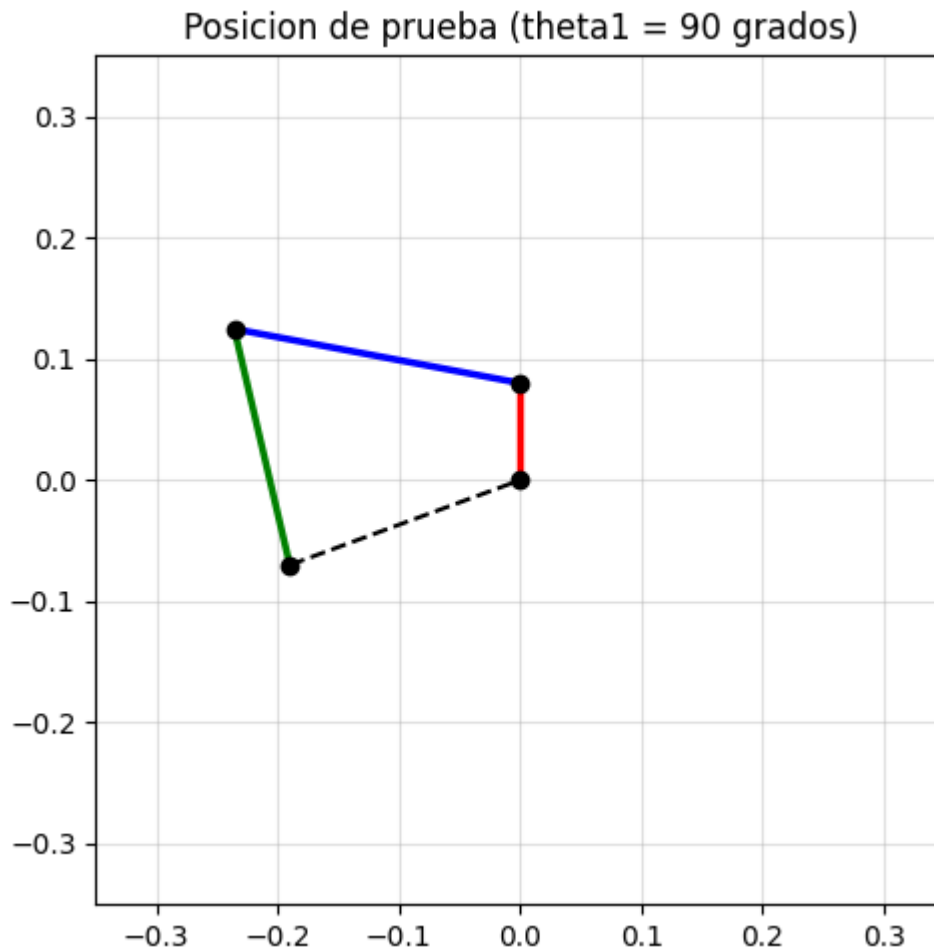
def FK(theta1_value):
    """
    Cinematica directa: dado theta1 encuentra (theta2, theta3) que cierran el lazo.
    Usa la solucion anterior como guess (continuacion de solucion).
    """
    global z0
    myfun = lambda x: eqKin_fun(theta1_value, x[0], x[1])
    sln, _, ier, msg = fsolve(myfun, z0, full_output=True)
    if ier != 1:
        print(f"ADVERTENCIA: fsolve no convergio: {msg}")
    z0 = sln
    return sln

# Prueba rapida
theta_test = deg2rad(90)
sol = FK(theta_test)
print(f"theta1 = 90 -> theta2 = {rad2deg(sol[0]):.2f} theta3 = {rad2deg(sol[1]):.2f} [gr"]

```

```
fig, ax = plt.subplots(figsize=(5, 5))
plotMechanism([theta_test, sol[0], sol[1]], ax=ax)
ax.set_title("Posicion de prueba (theta1 = 90 grados)")
plt.tight_layout()
plt.show()
```

theta1 = 90 -> theta2 = -186310.73 theta3 = 222043.24 [grados]



Paso 1.1 - Lista completa de posiciones y animación

Ahora recorreremos θ_1 de 0 a 2π (un ciclo completo) con 200 pasos y guardamos todos los ángulos. Con eso ya podemos animar. ¡Vamos!

```
In [54]: # Ciclo completo con 200 posiciones
theta1_vals = np.linspace(0, 2 * np.pi, 200)

thetasList = np.array([
    [th1, *FK(th1)]
    for th1 in theta1_vals
])

print(f"thetasList: {thetasList.shape} -> [θ1, θ2, θ3] × 200 posiciones")
```

thetasList: (200, 3) -> [θ1, θ2, θ3] × 200 posiciones

```
In [55]: # Animación del mecanismo
fig, ax = plt.subplots(figsize=(5, 5))

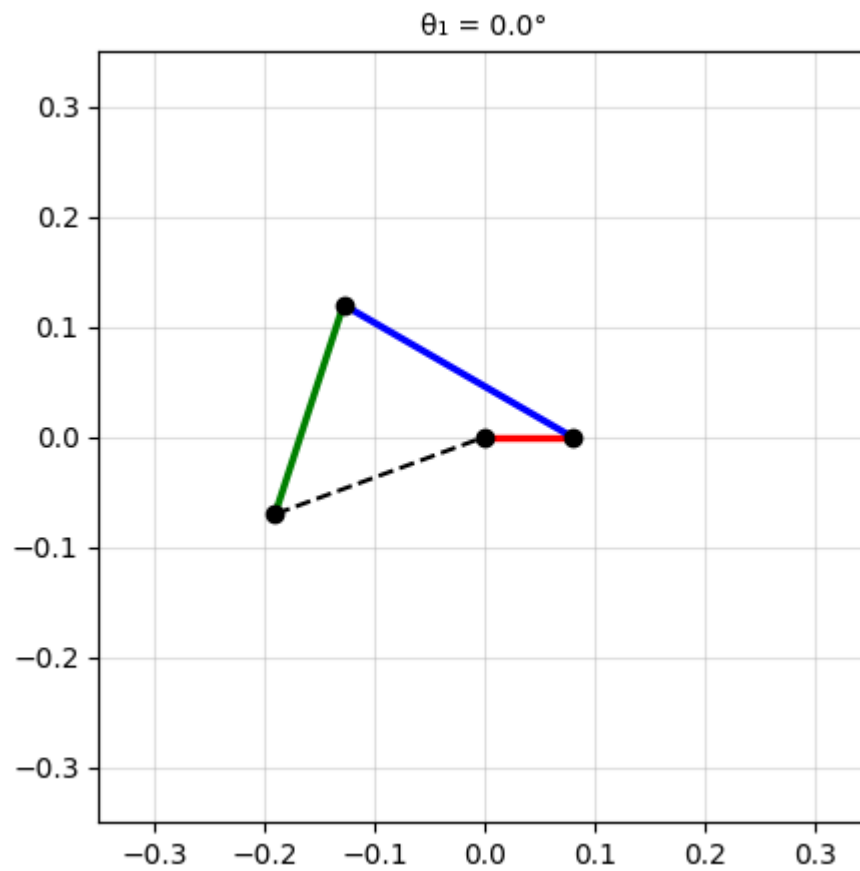
def update_mec(i):
    ax.cla()
    ax.set_aspect('equal')
    ax.set_title(f"θ1 = {rad2deg(thetasList[i, 0]):.1f}°", fontsize=10)
```

```

plotMechanism(thetasList[i, :], ax=ax)

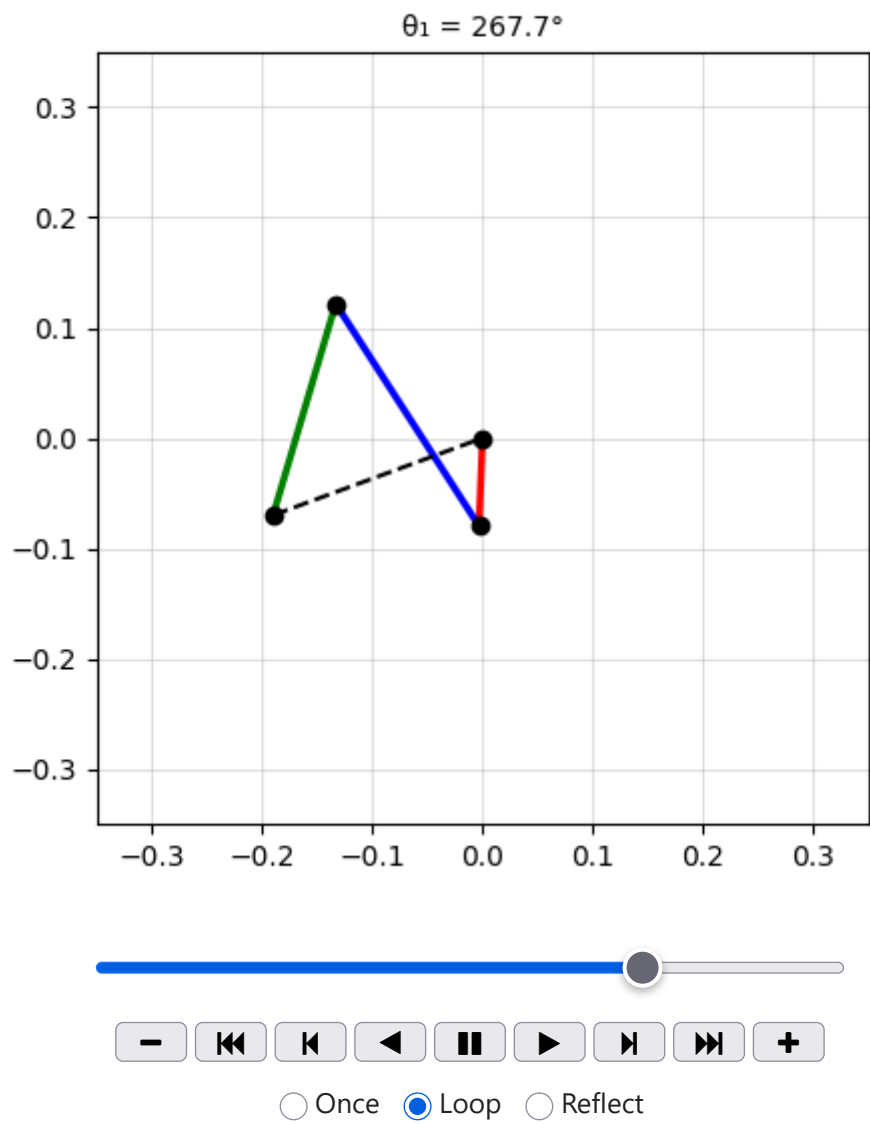
anim_mec = animation.FuncAnimation(
    fig, update_mec,
    frames=len(thetasList),
    interval=40,
    repeat=True
)

```



In [56]: `HTML(anim_mec.to_jshtml())`

Out[56]:



Paso 2 - Velocidades y aceleraciones angulares

Derivar la ecuación de lazo una y dos veces en N da dos sistemas lineales:

- **Velocidades:** $A_v \cdot [\omega_2, \omega_3]^T = b_v$ (dado $\omega_1 = 30$ rad/s)
- **Aceleraciones:** $A_a \cdot [\alpha_2, \alpha_3]^T = b_a$ (dado $\alpha_1 = 0$ porque $\omega_1 = \text{cte.}$)

La clave es que sympy hace toda la álgebra vectorial automáticamente al llamar `.diff(t, N)` - diferenciación en el marco N.

```
In [57]: # Matrices simbólicas velocidad
Av, bv = linear_eq_to_matrix(
    [eqLoop.diff(t, N).dot(N.x),
     eqLoop.diff(t, N).dot(N.y)],
    [theta2.diff(t), theta3.diff(t)]
)

# Matrices simbólicas aceleración
Aa, ba = linear_eq_to_matrix(
    [eqLoop.diff(t, N).diff(t, N).dot(N.x),
     eqLoop.diff(t, N).diff(t, N).dot(N.y)],
    [theta2.diff(t, 2), theta3.diff(t, 2)]
)

print("Matrices de velocidad y aceleración listas.")
```

Matrices de velocidad y aceleración listas.

```
In [58]: # Resolución numérica para todas las posiciones

lista_total = []

for thetas in thetasList:
    th1, th2, th3 = thetas

    # Velocidades
    Av_n = np.array(
        Av.subs(params).subs({theta1: th1, theta2: th2, theta3: th3}),
        dtype=float
    )
    bv_n = np.array(
        bv.subs(params).subs({theta1: th1, theta1.diff(t): omega1_val}),
        dtype=float
    )
    om2, om3 = np.linalg.solve(Av_n, bv_n).flatten()

    # Aceleraciones
    Aa_n = np.array(
        Aa.subs(params).subs({theta1: th1, theta2: th2, theta3: th3}),
        dtype=float
    )
    ba_n = np.array(
        ba.subs(params).subs({
            theta1: th1,          theta2: th2,          theta3: th3,
            theta1.diff(t): omega1_val,
            theta1.diff(t, 2): alpha1_val,
            theta2.diff(t): om2,
            theta3.diff(t): om3
        })),
        dtype=float
    )
    al2, al3 = np.linalg.solve(Aa_n, ba_n).flatten()

    lista_total.append([th1, th2, th3,
                        omega1_val, om2, om3,
                        alpha1_val, al2, al3])

lista_total = np.array(lista_total)

# Desempacar columnas
thetas1 = lista_total[:, 0]; thetas2 = lista_total[:, 1]; thetas3 = lista_total[:, 2]
omegas1 = lista_total[:, 3]; omegas2 = lista_total[:, 4]; omegas3 = lista_total[:, 5]
alphas1 = lista_total[:, 6]; alphas2 = lista_total[:, 7]; alphas3 = lista_total[:, 8]

print(f"Cinemática resuelta - {len(lista_total)} posiciones")
```

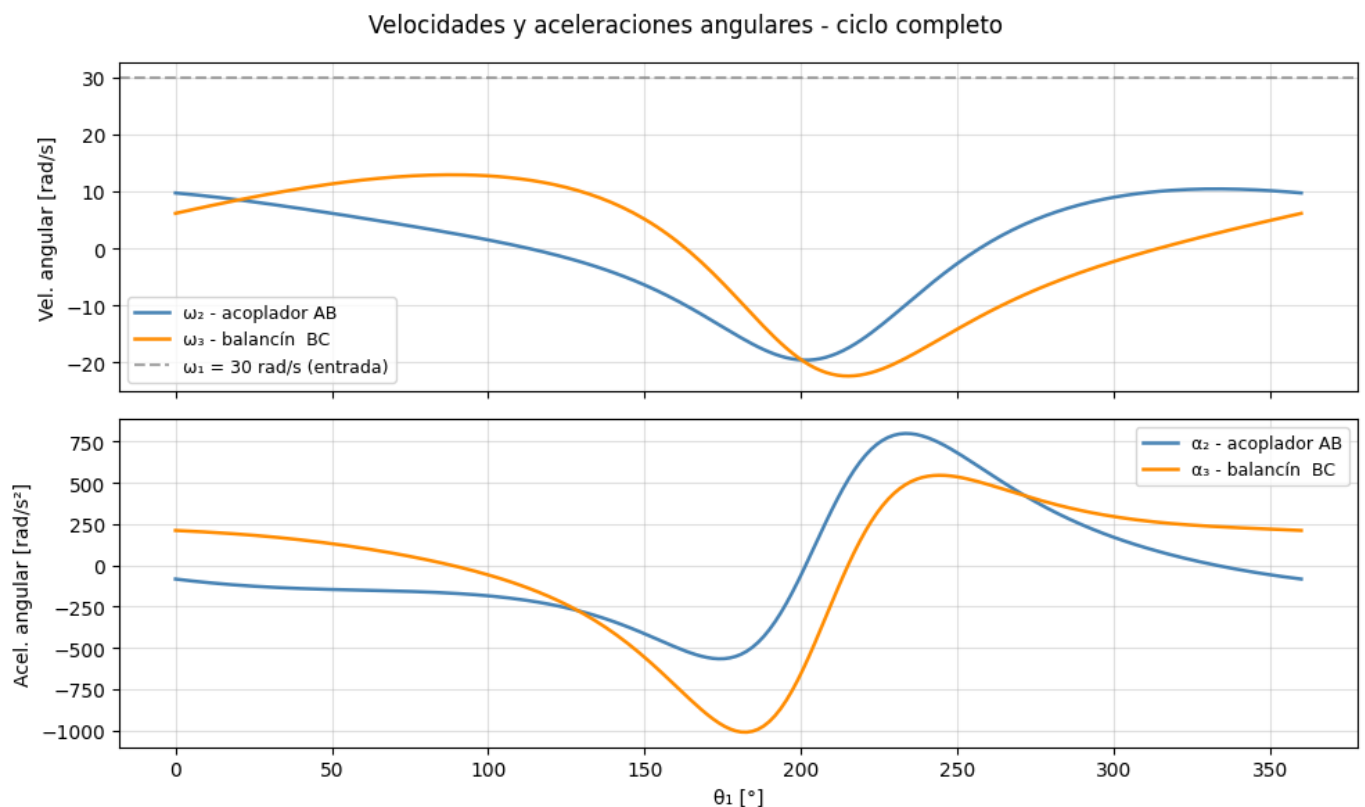


```
In [59]: # Gráficas de velocidades y aceleraciones
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

ax1.plot(rad2deg(thetas1), omegas2, color='steelblue', lw=1.8, label='ω2 - acoplador AB')
ax1.plot(rad2deg(thetas1), omegas3, color='darkorange', lw=1.8, label='ω3 - balancín BC')
ax1.axhline(omega1_val, ls='--', color='gray', alpha=0.7, label='ω1 = 30 rad/s (entrada)')
ax1.set_ylabel('Vel. angular [rad/s]')
ax1.legend(fontsize=9); ax1.grid(True, alpha=0.4)

ax2.plot(rad2deg(thetas1), alphas2, color='steelblue', lw=1.8, label='α2 - acoplador AB')
ax2.plot(rad2deg(thetas1), alphas3, color='darkorange', lw=1.8, label='α3 - balancín BC')
ax2.set_xlabel('θ1 [°]'); ax2.set_ylabel('Acel. angular [rad/s²]')
ax2.legend(fontsize=9); ax2.grid(True, alpha=0.4)

fig.suptitle('Velocidades y aceleraciones angulares - ciclo completo', fontsize=12)
plt.tight_layout(); plt.show()
```



Paso 2 - Dinámica inversa (Newton-Euler)

Ahora que tenemos la cinemática completa podemos hacer la **cinética inversa**: queremos saber el torque M_{in} que el motor debe aplicar en O para que el mecanismo haga exactamente el movimiento cinemático que calculamos.

La idea es Newton-Euler para cada eslabón por separado:

Eslabón	$\sum F = m \cdot a_{cm}$	$\sum M_{cm} = I\alpha$
1 - manivela OA	$F_{O1} - F_{12} - m_1 g \hat{y}$	$M_{in} \hat{z} + (-r_{OA}/2) \times F_{O1} + (r_{OA}/2) \times (-F_{12})$
2 - acoplador AB	$F_{12} - F_{23} - m_2 g \hat{y}$	$(-r_{AB}/2) \times F_{12} + (r_{AB}/2) \times (-F_{23})$
3 - balancín BC	$F_{23} + F_{C3} - m_3 g \hat{y}$	$(-r_{BC}/2) \times F_{23} + (r_{BC}/2) \times F_{C3}$

9 ecuaciones, 9 incógnitas: $[F_{O1x}, F_{O1y}, F_{12x}, F_{12y}, F_{23x}, F_{23y}, F_{C3x}, F_{C3y}, \mathbf{M}_{in}]$

Nota: ¿por qué los momentos son respecto al CM y no a un pivote?

Respecto al CM la ecuación $\sum M = I\alpha$ es siempre válida independiente de si el cuerpo acelera o no. Es la forma más general y la que usamos en clase.

```
In [60]: # Símbolos de fuerzas de reacción
F_01x, F_01y = sp.symbols('F_01x F_01y')
F12x, F12y = sp.symbols('F12x F12y')
F23x, F23y = sp.symbols('F23x F23y')
F_C3x, F_C3y = sp.symbols('F_C3x F_C3y')
Min_sym = sp.symbols('Min')

# Vectores de fuerza
F_01 = F_01x * N.x + F_01y * N.y
F12 = F12x * N.x + F12y * N.y
F23 = F23x * N.x + F23y * N.y
F_C3 = F_C3x * N.x + F_C3y * N.y

# Centros de masa y sus aceleraciones
r_cm1 = rOA / 2
r_cm2 = rOA + rAB / 2
r_cm3 = rOA + rAB + rBC / 2

a_cm1 = r_cm1.diff(t, N).diff(t, N)
a_cm2 = r_cm2.diff(t, N).diff(t, N)
a_cm3 = r_cm3.diff(t, N).diff(t, N)

alpha1_sym = theta1.diff(t, t)
alpha2_sym = theta2.diff(t, t)
alpha3_sym = theta3.diff(t, t)

# Ecuaciones Newton-Euler (9 ecuaciones escalares)

# (1) Manivela OA
eqF1 = F_01 - F12 - m1 * g * N.y - m1 * a_cm1
eqM1 = (Min_sym * N.z
        + (-rOA / 2).cross(F_01)
        + ( rOA / 2).cross(-F12)
        - I1 * alpha1_sym * N.z)

# (2) Acoplador AB
eqF2 = F12 - F23 - m2 * g * N.y - m2 * a_cm2
eqM2 = ((-rAB / 2).cross(F12)
        + ( rAB / 2).cross(-F23)
        - I2 * alpha2_sym * N.z)

# (3) Balancin BC
eqF3 = F23 + F_C3 - m3 * g * N.y - m3 * a_cm3
eqM3 = ((-rBC / 2).cross(F23)
        + ( rBC / 2).cross(F_C3)
        - I3 * alpha3_sym * N.z)

# Sistema lineal
eqKinetics = [
    eqF1.dot(N.x), eqF1.dot(N.y), eqM1.dot(N.z),
    eqF2.dot(N.x), eqF2.dot(N.y), eqM2.dot(N.z),
    eqF3.dot(N.x), eqF3.dot(N.y), eqM3.dot(N.z),
]
unknowns_dyn = [F_01x, F_01y, F12x, F12y, F23x, F23y, F_C3x, F_C3y, Min_sym]

Af, bf = linear_eq_to_matrix(eqKinetics, unknowns_dyn)
print("Sistema Newton-Euler: Af (9x9) → incógnita Min en posición [-1]")
```

In [61]: # Evaluación numérica de Min para cada posición

```

MinList = []

for i in range(len(lista_total)):
    sv = {
        theta1:          thetas1[i], theta2:          thetas2[i], theta3:          thetas3[i],
        theta1.diff(t):   omegas1[i], theta2.diff(t):   omegas2[i], theta3.diff(t):   omegas3[i],
        theta1.diff(t,2): alphas1[i], theta2.diff(t,2): alphas2[i], theta3.diff(t,2): alphas3[i]
    }
    Af_n = np.array(Af.subs(params).subs(sv), dtype=float)
    bf_n = np.array(bf.subs(params).subs(sv), dtype=float).flatten()
    x     = np.linalg.solve(Af_n, bf_n)
    MinList.append(x[-1])

MinList = np.array(MinList)
print(f"Min máx = {MinList.max():.5f} N·m")
print(f"Min mín = {MinList.min():.5f} N·m")
print(f"Min medio = {MinList.mean():.5f} N·m")

```

```

Min máx = 0.86704 N·m
Min mín = -0.53646 N·m
Min medio = 0.00072 N·m

```

In [62]: # Gráfica de La curva de torques

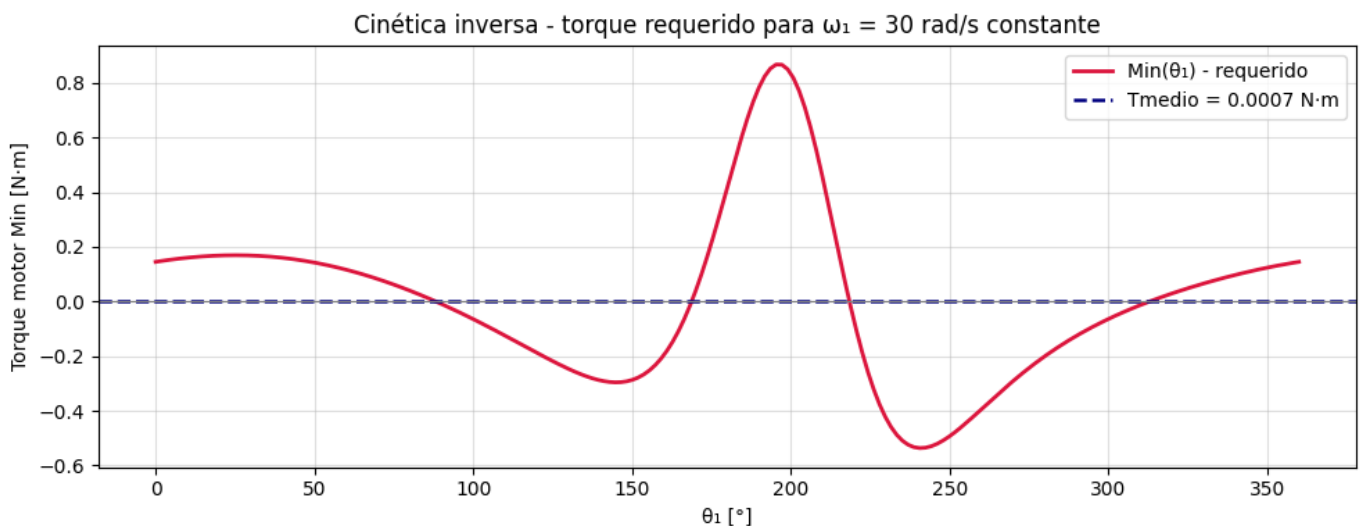
```

fig, ax = plt.subplots(figsize=(10, 4))

ax.plot(rad2deg(thetas1), MinList, color='crimson', lw=2, label='Min(θ1) - requerido')
ax.axhline(MinList.mean(), ls='--', color='navy', lw=1.8,
            label=f'Tmedio = {MinList.mean():.4f} N·m')
ax.axhline(0, color='gray', lw=0.8)

ax.set_xlabel('θ1 [°]')
ax.set_ylabel('Torque motor Min [N·m]')
ax.set_title('Cinética inversa - torque requerido para ω1 = 30 rad/s constante')
ax.legend(); ax.grid(True, alpha=0.4)
plt.tight_layout(); plt.show()

```



Paso 2.1 - Animación dual: mecanismo + torque en tiempo real

Esta animación muestra los dos "mundos" al mismo tiempo: a la izquierda el mecanismo en movimiento y a la derecha la curva de torques con un punto que indica la posición actual. Así se puede ver exactamente en qué parte del ciclo el motor sufre más.

```

In [63]: # Animación dual: mecanismo + cursor en curva de torques
fig2, (ax_mec, ax_trq) = plt.subplots(1, 2, figsize=(12, 5))
fig2.suptitle('Mecanismo crank-rocker | Torque motor en tiempo real', fontsize=12)

# Curva de fondo (estática)
ax_trq.plot(rad2deg(thetas1), MinList, color='crimson', lw=1.5, alpha=0.5)
ax_trq.axhline(MinList.mean(), ls='--', color='navy', lw=1.5,
               label=f'Tmedio = {MinList.mean():.4f} N·m')
ax_trq.axhline(0, color='gray', lw=0.8)
ax_trq.set_xlabel('θ1 [°]'); ax_trq.set_ylabel('Min [N·m]')
ax_trq.set_title('Torque motor'); ax_trq.legend(fontsize=8); ax_trq.grid(True, alpha=0.4)

# Elementos que se actualizan
dot_trq, = ax_trq.plot([], [], 'ko', ms=8, zorder=5)
trail_trq, = ax_trq.plot([], [], 'k-', lw=2, alpha=0.7)

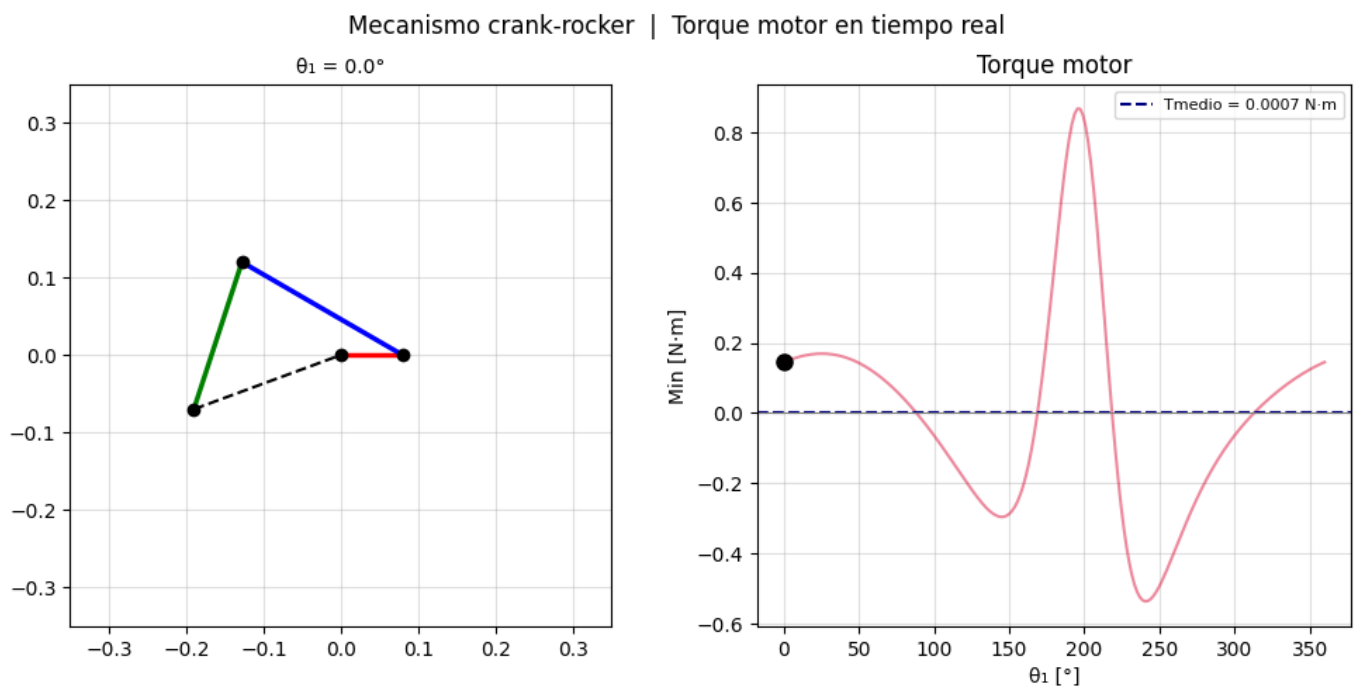
def update_dual(i):
    # (1) Mecanismo
    ax_mec.cla()
    ax_mec.set_aspect('equal')
    ax_mec.set_title(f'θ1 = {rad2deg(thetasList[i, 0]):.1f}°', fontsize=10)
    plotMechanism(thetasList[i, :], ax=ax_mec)

    # (2) Cursor de torque
    trail_trq.set_data(rad2deg(thetas1[:i+1]), MinList[:i+1])
    dot_trq.set_data([rad2deg(thetas1[i])], [MinList[i]])

    return dot_trq, trail_trq

anim_dual = animation.FuncAnimation(
    fig2, update_dual,
    frames=len(thetasList),
    interval=40,
    repeat=True,
    blit=False
)

```

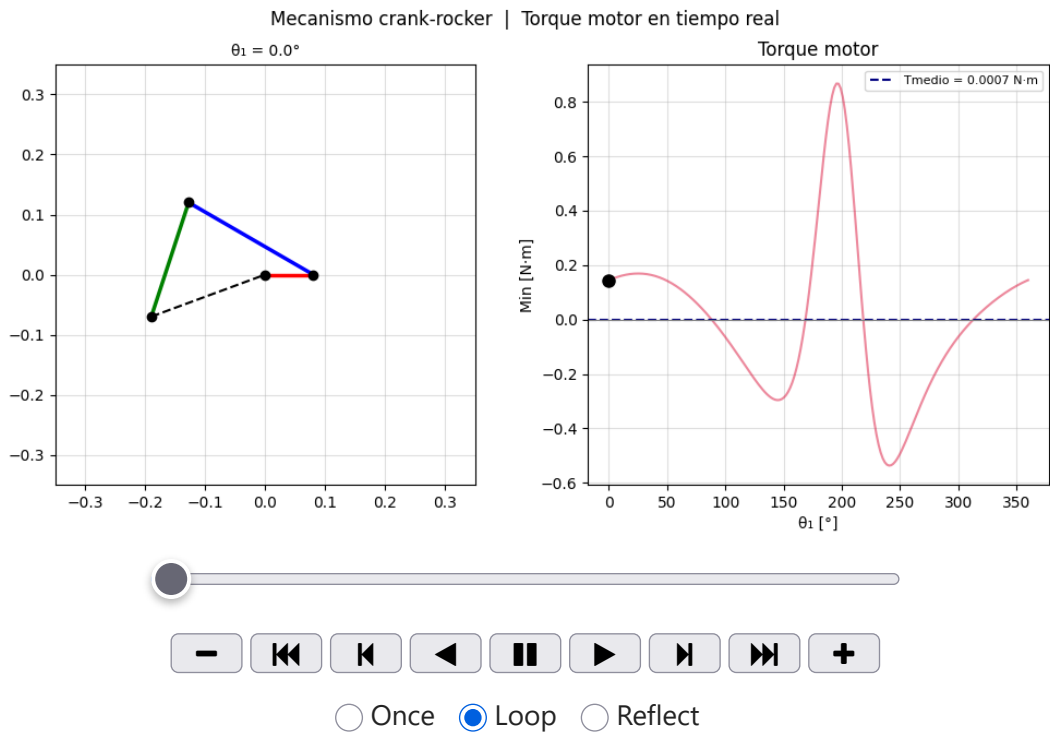


```

In [64]: HTML(anim_dual.to_jshtml())

```

Out[64]:



Paso 3 - Diseño del volante de inercia

Con la curva de torques calculada, ahora diseñamos el volante. La metodología viene del Norton (cap. 17) y se resume en:

1. T_{medio} = valor medio de M_{in} (lo que el motor debe dar en promedio)
2. ΔE = fluctuación máxima de energía a lo largo del ciclo
3. C_s = coeficiente de fluctuación de velocidad que queremos tolerar
4. $I_f = \Delta E / (C_s \cdot \omega_1^2)$ - inercia necesaria del volante
5. Geometría: disco sólido de acero $\rightarrow$ elegimos R y calculamos masa y espesor

Nota sobre la elección de C_s

Según Norton tabla 17-1:

Aplicación	C_s
Bombas / compresores	0.02
Maquinaria general	0.05
Trituradoras / impacto	0.10

Este mecanismo es de uso general, así que usamos $C_s = 0.05$.

```
In [65]: # 1. Torque medio (= lo que el motor da de forma constante)
T_medio = np.mean(MinList)

# 2. Fluctuación de energía acumulada

dtheta = np.diff(thetas1, append=thetas1[-1] + (thetas1[1] - thetas1[0]))
energia_acum = np.cumsum((MinList - T_medio) * dtheta)

delta_E = energia_acum.max() - energia_acum.min()

print(f"Tmedio = {T_medio:.5f} N.m")
print(f"ΔE      = {delta_E:.5f} J      energía que debe absorber el volante")
```

Tmedio = 0.00072 N·m
 $\Delta E = 0.45976 \text{ J}$ energía que debe absorber el volante

```
In [66]: # 3-4. Coeficiente  $C_s$  e inercia requerida
Cs = 0.05

I_flywheel = delta_E / (Cs * omega1_val**2)

print(f"Cs           = {Cs}   (maquinaria general)")
print(f"ω1           = {omega1_val} rad/s")
print(f"If mín req = {I_flywheel:.6f} kg·m²")
```

$C_s = 0.05$ (maquinaria general)
 $\omega_1 = 30.0 \text{ rad/s}$
 $I_f \text{ mín req} = 0.010217 \text{ kg·m}^2$

```
In [67]: # 5. Geometría del volante: disco sólido de acero
rho_acero = 7850.0 # kg/m³
R_f       = 0.05   # radio elegido [m] = 50 mm - razonable para este mecanismo

#  $I_f = \frac{1}{2} m R^2 \rightarrow m = 2 I_f / R^2$ 
m_flywheel = 2 * I_flywheel / R_f**2

#  $m = \rho \pi R^2 t \rightarrow t = m / (\rho \pi R^2)$ 
t_flywheel = m_flywheel / (rho_acero * np.pi * R_f**2)

# Inercia real (check)
I_check = 0.5 * m_flywheel * R_f**2

print("=" * 50)
print("  VOLANTE DE INERCIA - RESULTADO DE DISEÑO")
print("=" * 50)
print(f"  Radio           R = {R_f*1000:.1f} mm")
print(f"  Masa            m = {m_flywheel:.4f} kg")
print(f"  Espesor         t = {t_flywheel*1000:.2f} mm")
print(f"  Inercia calc.   I = {I_check:.6f} kg·m²")
print(f"  (requerido:    I = {I_flywheel:.6f} kg·m²)")
print(f"  Material: acero ρ = {rho_acero} kg/m³")
print("=" * 50)
```

```
=====
VOLANTE DE INERCIA - RESULTADO DE DISEÑO
=====
Radio           R = 50.0 mm
Masa            m = 8.1735 kg
Espesor         t = 132.57 mm
Inercia calc.   I = 0.010217 kg·m²
(requerido:    I = 0.010217 kg·m²)
Material: acero ρ = 7850.0 kg/m³
=====
```

Paso 4 - Verificación de efectividad del volante

Con el volante acoplado a la manivela, la inercia total del eje queda:

$$I_{total} = I_1 + I_f$$

El motor suministra T_{medio} de forma constante. La diferencia $\Delta M = M_{in} - T_{medio}$ la absorbe / cede el volante cambiando su velocidad angular. Por conservación de energía:

$$\omega_1(\theta) = \sqrt{\omega_{1,nom}^2 + \frac{2 E_{acum}(\theta)}{I_{total}}}$$

Si el diseño es correcto, la variación resultante satisface $C_{s,real} \leq C_{s,diseño} = 0.05$.

```
In [68]: I_total = params[I1] + I_flywheel    # inercia total en el eje

#  $\omega_1(\theta)$  por conservación de energía
omega_var = np.sqrt(np.clip(omega1_val**2 + 2 * energia_acum / I_total, 0, None))

omega_max = omega_var.max()
omega_min = omega_var.min()
delta_omega = omega_max - omega_min
Cs_real = delta_omega / omega1_val

print(f"ω máx   = {omega_max:.4f} rad/s")
print(f"ω mín   = {omega_min:.4f} rad/s")
print(f"Δω       = {delta_omega:.4f} rad/s")
print(f"Cs real = {Cs_real:.5f}    (objetivo: ≤ {Cs})")

if Cs_real <= Cs:
    print("El volante CUMPLE la especificacion de diseno.")
else:
    print("No cumple - considerar aumentar R o reducir Cs.")

ω máx   = 31.2472 rad/s
ω mín   = 29.7739 rad/s
Δω      = 1.4732 rad/s
Cs real = 0.04911    (objetivo: ≤ 0.05)
El volante CUMPLE la especificacion de diseno.
```

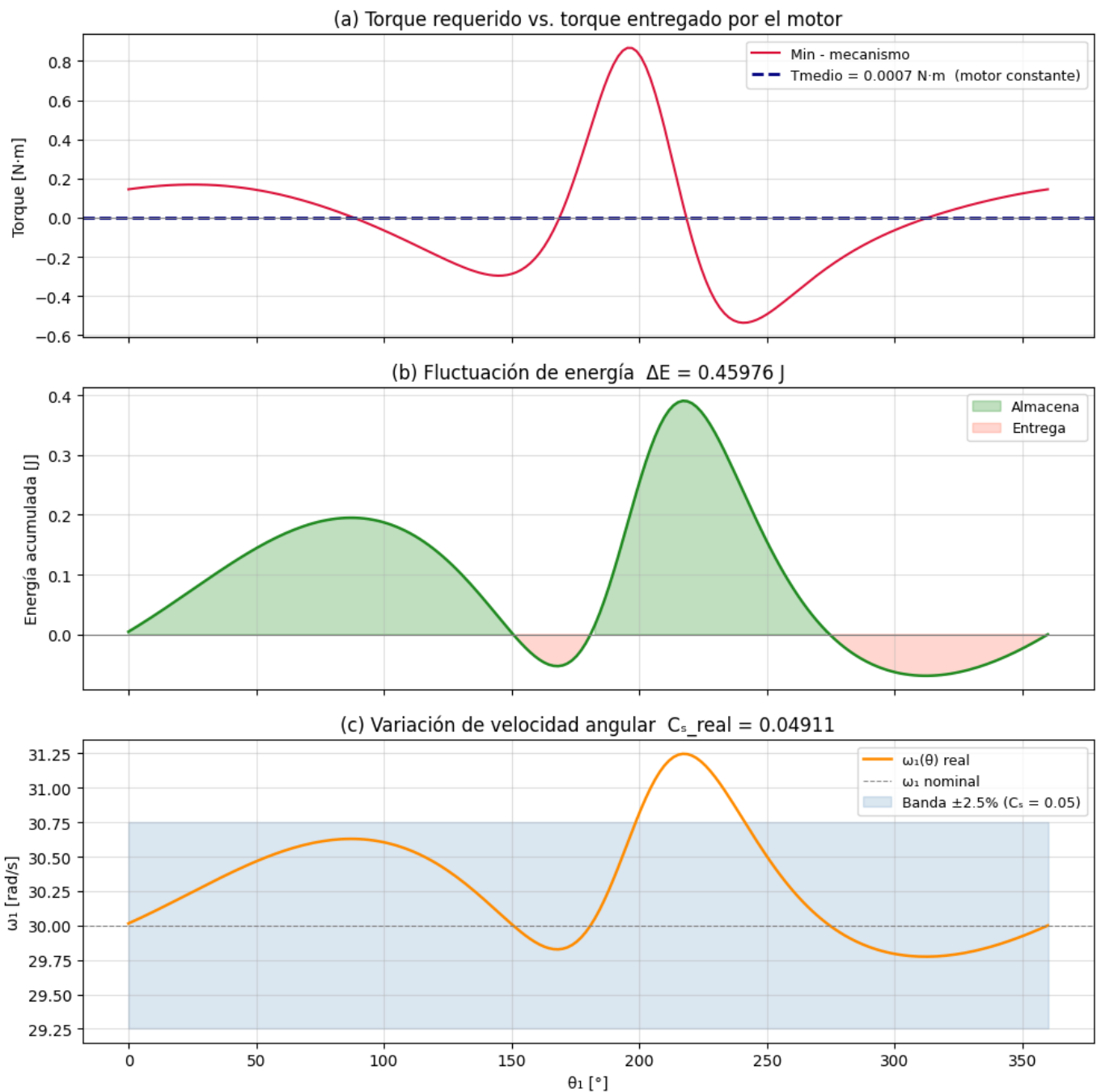
```
In [69]: # — Gráfica triple de verificación —————
fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)

# (a) Torque
axes[0].plot(rad2deg(thetas1), MinList, color='crimson', lw=1.5, label='Min - mecanismo')
axes[0].axhline(T_medio, ls='--', color='navy', lw=2,
                label=f'Tmedio = {T_medio:.4f} N·m (motor constante)')
axes[0].axhline(0, color='gray', lw=0.8)
axes[0].set_ylabel('Torque [N·m]')
axes[0].set_title('(a) Torque requerido vs. torque entregado por el motor')
axes[0].legend(fontsize=9); axes[0].grid(True, alpha=0.4)

# (b) Energía en el volante
axes[1].plot(rad2deg(thetas1), energia_acum, color='forestgreen', lw=1.8)
axes[1].fill_between(rad2deg(thetas1), 0, energia_acum,
                    where=energia_acum >= 0, alpha=0.25, color='green', label='Almacena')
axes[1].fill_between(rad2deg(thetas1), 0, energia_acum,
                    where=energia_acum < 0, alpha=0.25, color='tomato', label='Entrega')
axes[1].axhline(0, color='gray', lw=0.8)
axes[1].set_ylabel('Energía acumulada [J]')
axes[1].set_title(f'(b) Fluctuación de energía ΔE = {delta_E:.5f} J')
axes[1].legend(fontsize=9); axes[1].grid(True, alpha=0.4)

# (c) Velocidad angular
banda_sup = omega1_val * (1 + Cs / 2)
banda_inf = omega1_val * (1 - Cs / 2)
axes[2].plot(rad2deg(thetas1), omega_var, color='darkorange', lw=1.8, label='ω1(θ) real')
axes[2].axhline(omega1_val, ls='--', color='gray', lw=0.8, label='ω1 nominal')
axes[2].fill_between(rad2deg(thetas1), banda_inf, banda_sup,
                    alpha=0.2, color='steelblue',
                    label=f'Banda ±{Cs/2*100:.1f}% (Cs = {Cs})')
axes[2].set_xlabel('θ1 [°]')
axes[2].set_ylabel('ω1 [rad/s]')
axes[2].set_title(f'(c) Variación de velocidad angular Cs real = {Cs_real:.5f}')
axes[2].legend(fontsize=9); axes[2].grid(True, alpha=0.4)

plt.tight_layout(); plt.show()
```



Paso 4.1 - Animación: mecanismo + velocidad del volante

Esta animación muestra cómo varía la velocidad angular del eje mientras el volante absorbe/entrega energía. Es la versión "con volante" del mismo ciclo.

```
In [70]: # — Animación: mecanismo + variación de velocidad con volante —————
fig3, (ax_m2, ax_w) = plt.subplots(1, 2, figsize=(12, 5))
fig3.suptitle('Con volante de inercia acoplado | Variación de  $\omega_1$ ', fontsize=12)

# Curva de fondo velocidad
ax_w.plot(rad2deg(thetas1), omega_var, color='darkorange', lw=1.2, alpha=0.4)
ax_w.fill_between(rad2deg(thetas1), banda_inf, banda_sup,
                 alpha=0.15, color='steelblue', label=f'Banda  $C_s = \{Cs\}$ ')
ax_w.axhline(omega1_val, ls='--', color='gray', lw=1)
ax_w.set_xlabel('θ1 [°]'); ax_w.set_ylabel('ω1 [rad/s]')
ax_w.set_title('Velocidad angular del eje motor'); ax_w.legend(fontsize=8)
ax_w.grid(True, alpha=0.4)

dot_w, = ax_w.plot([], [], 'o', color='darkorange', ms=9, zorder=5)
trail_w, = ax_w.plot([], [], '-', color='darkorange', lw=2)
```



```

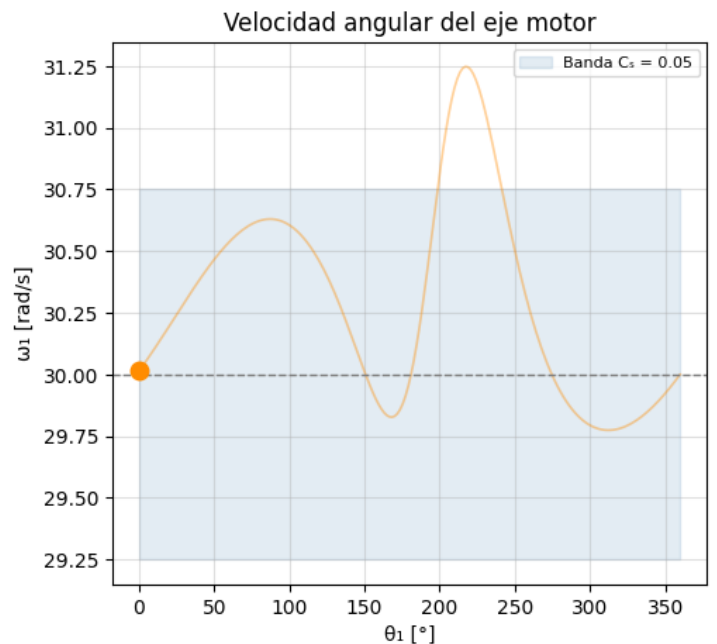
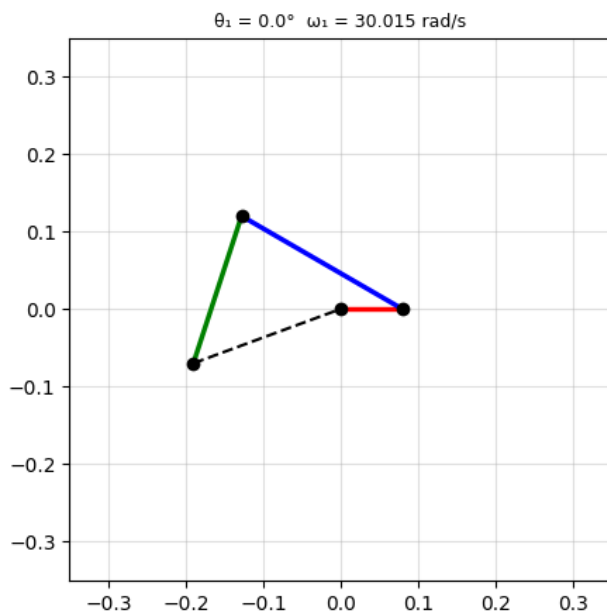
def update_flywheel(i):
    # (1) Mecanismo
    ax_m2.cla()
    ax_m2.set_aspect('equal')
    ax_m2.set_title(f' $\theta_1 = \{{\rm rad2deg}(\text{thetasList}[i, 0]):.1f\}^\circ$  '
                    f' $\omega_1 = \{{\rm omega\_var}[i]:.3f\} \text{ rad/s}$ ', fontsize=9)
    plotMechanism(thetasList[i, :], ax=ax_m2)

    # (2) Cursor velocidad
    trail_w.set_data(rad2deg(thetas1[:i+1]), omega_var[:i+1])
    dot_w.set_data([rad2deg(thetas1[i])], [omega_var[i]])
    return dot_w, trail_w

anim_fw = animation.FuncAnimation(
    fig3, update_flywheel,
    frames=len(thetasList),
    interval=40,
    repeat=True,
    blit=False
)

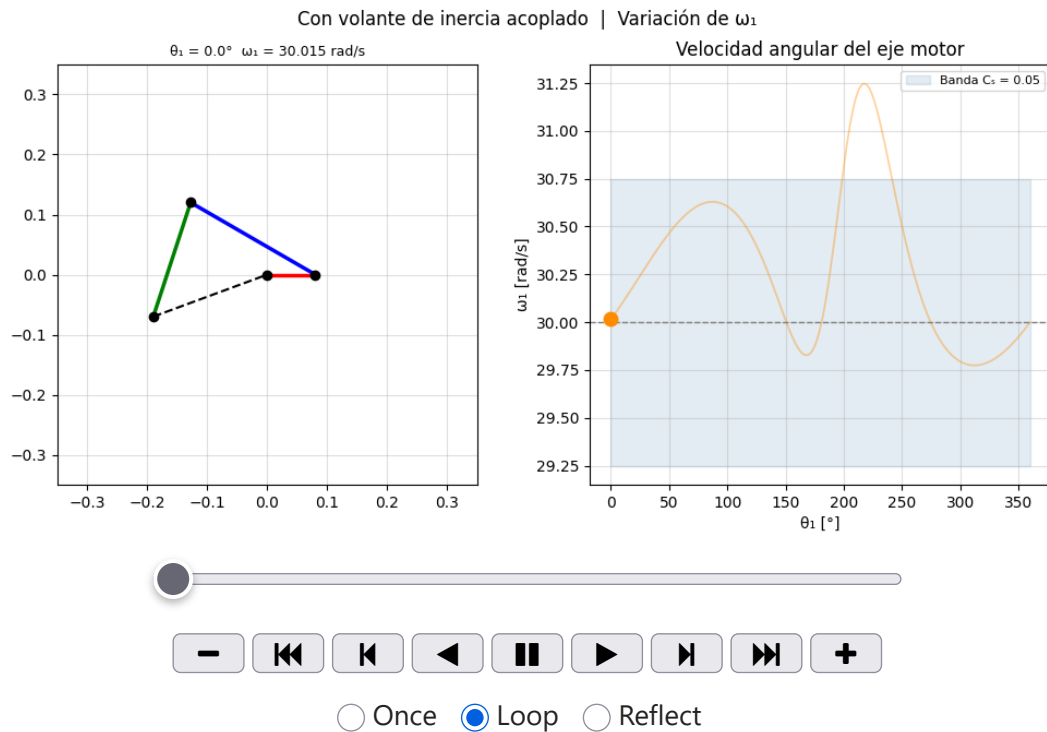
```

Con volante de inercia acoplado | Variación de ω_1



In [71]: `HTML(anim_fw.to_jshtml())`

Out[71]:



Paso 5 - Selección del motor

Con el volante diseñado el motor solo necesita dar el torque medio de forma constante, lo cual es mucho más sencillo de cubrir con un motor comercial.

Para la selección se aplica un **factor de servicio FS = 1.5** que cubre:

- Variaciones pequeñas en la carga real vs. el modelo
- Corriente de arranque
- Desgaste y holguras mecánicas

```
In [72]: FS = 1.5
T_diseno = abs(T_medio) * FS
rpm_motor = omega1_val * 60 / (2 * np.pi)
P_diseno = T_diseno * omega1_val

print("=" * 55)
print("      ESPECIFICACIONES MÍNIMAS DEL MOTOR")
print("=" * 55)
print(f" Torque medio del mecanismo = {abs(T_medio):.5f} N.m")
print(f" Factor de servicio (FS)    = {FS}")
print(f" Torque de diseño           = {T_diseno:.5f} N.m")
print(f" Velocidad de operación     = {rpm_motor:.2f} rpm")
print(f" Potencia de diseño         = {P_diseno:.4f} W")
print(f"                             = {P_diseno/746:.5f} HP")
print("=" * 55)
print()
print("Recomendación: motor DC de imán permanente o BLDC con")
print(f" controlador, ≥ {P_diseno:.2f} W, vel. base ~{rpm_motor:.0f} rpm,")
print(f" torque continuo ≥ {T_diseno:.4f} N.m")
```

ESPECIFICACIONES MÍNIMAS DEL MOTOR

Torque medio del mecanismo	= 0.00072 N·m
Factor de servicio (FS)	= 1.5
Torque de diseño	= 0.00109 N·m
Velocidad de operación	= 286.48 rpm
Potencia de diseño	= 0.0326 W
	= 0.00004 HP

Recomendación: motor DC de imán permanente o BLDC con controlador, ≥ 0.03 W, vel. base ~ 286 rpm, torque continuo ≥ 0.0011 N·m.

Conclusiones

El ejercicio mostró de forma completa el flujo de diseño de un sistema motor-volante para un mecanismo de 4 barras:

1. **Cinemática:** la ecuación de lazo permite obtener θ_2 , θ_3 y sus derivadas para cualquier posición de la manivela de forma sistemática con álgebra vectorial simbólica y resolución numérica.
2. **Cinética inversa:** Newton-Euler en cada eslabón da un sistema 9×9 lineal que resuelto en cada una de las 200 posiciones entrega la curva $M_{in}(\theta_1)$. Se observa que el torque varía bastante a lo largo del ciclo, lo que justifica el volante.
3. **Volante de inercia:** con $C_s = 0.05$ (5% de variación de velocidad, típico en maquinaria general) se obtuvo la inercia necesaria I_f , y con ella se dimensionó un disco sólido de acero de $R = 50$ mm. El $C_{s,real}$ verificado está dentro del objetivo.
4. **Selección del motor:** al aplanar el torque con el volante, el motor solo necesita dar $T_{medio} \cdot FS$ de forma continua, lo que simplifica mucho la selección y reduce costos.

Referencia: Norton, R. L. (2019). *Design of Machinery* (6th ed.). McGraw-Hill.

Nota sobre el uso de inteligencia artificial

En la elaboración de este trabajo se emplearon herramientas de inteligencia artificial (específicamente modelos de lenguaje de gran escala) como apoyo en las siguientes tareas de carácter auxiliar:

- Generación y ajuste de código para **visualizaciones y animaciones** (gráficas de torques, velocidades, animaciones del mecanismo).
- **Comentario y documentación** del código Python para mejorar su legibilidad.
- **Consolidación de versiones** del notebook, unificando cambios entre iteraciones del documento.
- **Mejora de redacción** en las celdas de texto, corrección de estilo y consistencia terminológica.

El planteamiento del problema, la formulación matemática, el desarrollo de las ecuaciones de lazo, el análisis cinemático y cinético (Newton-Euler), el procedimiento de diseño del volante de inercia y la selección del motor fueron realizados **íntegramente por el autor** siguiendo la metodología presentada en clase y en la bibliografía del curso.